

Lesson 15 Structured Note: Converting Infix to Postfix Using a Stack

0. Current Position in the Course

You are now in **Lesson 15** of the stack curriculum.

Before this lesson, you already learned:

Lesson	Main idea
Lesson 1	A stack follows LIFO: Last In, First Out .
Lesson 2	An array stack uses <code>size</code> , <code>top</code> , and an array pointer.
Lesson 3	Array stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 4	A complete C array-stack program combines all array-stack functions.
Lesson 5	A linked-list stack uses nodes and a <code>top</code> pointer.
Lesson 6	Linked-list stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 7	A complete C linked-list stack uses <code>struct Node</code> , a global <code>top</code> pointer, <code>push</code> , <code>pop</code> , <code>display</code> , and <code>main</code> .
Lesson 8	A C++ class can be used to implement a linked-list stack more safely.
Lesson 9	Parenthesis matching was introduced as a stack application.
Lesson 10	Parenthesis matching was implemented using an array character stack.
Lesson 11	Parenthesis matching was implemented using a linked-list character stack.
Lesson 12	Bracket matching was extended to <code>()</code> , <code>[]</code> , and <code>{}</code> .
Lesson 13	You learned infix, prefix, postfix, precedence, and simple manual expression conversion.
Lesson 14	You learned associativity and unary operators.

Lesson 15 continues:

Phase 4: Expression Notation and Conversion

In this lesson, you start learning how to convert an expression from **infix notation** to **postfix notation** using a stack.

1. Lesson 15 Goal

By the end of this lesson, you should understand:

- What infix notation means.
 - What postfix notation means.
 - Why computers often prefer postfix notation.
 - Why a stack is useful for infix-to-postfix conversion.
 - How to scan an infix expression from left to right.
 - Why operands go directly to the postfix output.
 - Why operators are stored temporarily in a stack.
 - How operator precedence affects conversion.
 - Why equal precedence matters for left-to-right operators.
 - Why we pop operators from the stack before pushing a weaker or equal operator.
 - How to dry-run the conversion of `a+b*c-d/e`.
 - How to write a beginner C++ program for infix-to-postfix conversion.
 - What Method 1 means.
 - What Method 2 means.
 - Why Method 1 is easier for beginners.
 - The time and space complexity of infix-to-postfix conversion.
-

2. Quick Review: What Is an Expression?

An **expression** is a combination of operands and operators.

Example:

A + B

This expression has:

Symbol	Role
A	Operand
+	Operator
B	Operand

Operand

An **operand** is a value or variable used in an expression.

Examples:

A, B, C, x, y, 5, 9

Operator

An **operator** is a symbol that performs an operation.

Examples:

+ - * /

Simple meanings:

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division

3. Quick Review: Infix and Postfix

Lesson 15 focuses on converting from **infix** to **postfix**.

3.1 Infix Notation

In **infix notation**, the operator is written between the operands.

Structure:

Operand Operator Operand

Example:

A + B

Here, **+** is between **A** and **B**.

So this is infix notation.

Humans usually write mathematical expressions in infix form.

Examples:

A + B
A - B
A * B
A / B
A + B * C

3.2 Postfix Notation

In **postfix notation**, the operator is written after the operands.

Structure:

Operand Operand Operator

Example:

A B +

This means the same thing as:

A + B

The operator **+** comes after **A** and **B**.

So this is postfix notation.

Examples:

Infix	Postfix
A + B	A B +
A - B	A B -
A * B	A B *
A / B	A B /

Important idea:

Infix and postfix can represent the same expression.
Only the position of the operator changes.

4. What Problem Are We Solving?

Humans usually write expressions in infix form.

Example:

a + b * c

But computers can evaluate postfix expressions more easily.

The postfix form of this expression is:

a b c * +

So Lesson 15 asks this question:

How can we convert an infix expression into postfix using a stack?

Example:

Infix: a + b * c
Postfix: a b c * +

Why is the answer a b c * + ?

Because multiplication has higher precedence than addition.

So:

$a + b * c$

really means:

$a + (b * c)$

In postfix, $b * c$ becomes:

$b c *$

Then we add a :

$a b c * +$

5. Why Do We Need a Stack?

In infix expressions, operators do not always execute in the order we see them.

Example:

$a + b * c$

When we scan from left to right, we see $+$ before $*$.

But $*$ must be done first because multiplication has higher precedence.

So the $+$ operator must wait.

This is where the stack helps.

The stack temporarily stores operators until it is safe to send them to the postfix output.

Simple idea:

Operands go directly to output.
Operators wait in the stack when needed.

The stack is not the final answer.

The stack is a waiting area for operators.

6. The Two Things We Build During Conversion

While scanning the infix expression, we build two things:

1. Postfix output
2. Operator stack

6.1 Postfix Output

The postfix output stores the final converted expression.

Example:

```
postfix = abc*+
```

6.2 Operator Stack

The operator stack stores operators temporarily.

Operators include:

```
+ - * /
```

Example stack:

```
+ *
```

This means the operators are waiting until they can be moved to the postfix output.

7. Operator Precedence in Lesson 15

Precedence means priority.

It answers this question:

Which operator should be handled first?

For Lesson 15, we use this simple precedence table:

Operator	Precedence	Meaning
<code>*</code> , <code>/</code>	2	Higher priority
<code>+</code> , <code>-</code>	1	Lower priority

So:

`*` and `/` are stronger than `+` and `-`

Example:

`a + b * c`

The `*` operator has higher precedence than `+`.

So multiplication must happen before addition.

That is why the postfix form is:

`abc*+`

not:

`ab+c*`

8. Method 1: Operands Direct, Operators in Stack

Lesson 15 mainly uses this beginner-friendly method.

The idea is simple:

If the symbol is an operand, send it directly to postfix.
If the symbol is an operator, use the stack.

This is called Method 1 in this note.

9. Rule 1: If the Symbol Is an Operand

If the current symbol is an operand, send it directly to the postfix output.

Operands are things like:

a, b, c, x, y, 5, 9

Example:

Expression:

a+b

First symbol:

a

a is an operand.

So we add it directly to postfix:

postfix = a

We do not push operands into the stack in Method 1.

Important rule:

Operand -> output directly

10. Rule 2: If the Symbol Is an Operator

If the current symbol is an operator, we compare it with the operator on top of the stack.

Operators are:

+ - * /

There are three main cases.

Case 1: Stack Is Empty

If the operator stack is empty, push the current operator.

Example:

Expression:

a+b

After reading **a**:

```
postfix = a
stack = empty
```

Now we read **+**.

The stack is empty.

So we push **+**:

```
stack = +
postfix = a
```

Case 2: Current Operator Has Higher Precedence

If the current operator has higher precedence than the stack top, push it.

Example:

Expression:

```
a+b*c
```

At some point:

```
postfix = ab  
stack = +
```

Now we read `*`.

Compare:

```
stack top = +  
current operator = *
```

Precedence:

```
+ has precedence 1  
* has precedence 2
```

Since `*` is stronger than `+`, push `*`.

Now:

```
stack = + *  
postfix = ab
```

The `*` must happen before `+`, so it stays above `+` in the stack.

Case 3: Current Operator Has Lower or Equal Precedence

If the current operator has lower or equal precedence than the stack top, pop from the stack into postfix.

Keep popping while the stack top has greater or equal precedence.

Then push the current operator.

The important condition is:

```
while (!st.empty() && precedence(st.top()) >= precedence(sym))
```

Meaning:

While the stack is not empty
and the operator on the stack is stronger than or equal to the current operator,
pop it into the postfix output.

Then push the current operator.

11. Why Do We Use Greater Than or Equal To?

The condition uses:

`>=`

not just:

`>`

This matters because `+`, `-`, `*`, and `/` are left-to-right operators in this beginner lesson.

Example: `a-b-c`

Expression:

`a - b - c`

Subtraction is left-to-right.

So the expression means:

`(a - b) - c`

Correct postfix:

```
ab-c-
```

When we see the second `-`, the first `-` is already in the stack.

The two operators have equal precedence.

Because subtraction is left-to-right, the first `-` must be handled before the second `-`.

So we pop the first `-` before pushing the second `-`.

That is why we use:

```
precedence(st.top()) >= precedence(sym)
```

Simple idea:

```
For left-to-right operators, equal precedence means pop first.
```

12. Main Algorithm in Simple Words

To convert an infix expression to postfix:

1. Create an empty stack for operators.
2. Create an empty postfix output string.
3. Scan the infix expression from left to right.
4. For each symbol:
 5. If it is an operand, add it directly to postfix.
 6. If it is an operator:
 - Pop from the stack while the stack top has greater or equal precedence.
 - Then push the current operator.
7. After scanning the full expression, pop all remaining operators from the stack into postfix.
8. The postfix string is the final answer.

Short version:

Operand -> output
Operator -> stack
End -> pop remaining stack operators

13. Algorithm as Pseudocode

```
infixToPostfix(expression):  
    create empty stack  
    create empty postfix string  
  
    for each symbol in expression:  
        if symbol is an operand:  
            add symbol to postfix  
        else if symbol is an operator:  
            while stack is not empty AND precedence(stack top) >=  
precedence(symbol):  
                pop stack top and add it to postfix  
            push symbol onto stack  
  
    while stack is not empty:  
        pop stack top and add it to postfix  
  
    return postfix
```

14. Very Simple Example: `a+b`

Convert:

`a+b`

Expected postfix:

`ab+`

Dry run:

Step	Symbol	Action	Stack	Postfix
1	<code>a</code>	Operand, send to output	empty	<code>a</code>

Step	Symbol	Action	Stack	Postfix
2	+	Operator, stack empty, push it	+	a
3	b	Operand, send to output	+	ab
End	—	Pop remaining +	empty	ab+

Final answer:

a+b -> ab+

15. Example: a+b*c

Convert:

a+b*c

Expected postfix:

abc*+

Dry run:

Step	Symbol	Action	Stack	Postfix
1	a	Operand, send to output	empty	a
2	+	Stack empty, push +	+	a
3	b	Operand, send to output	+	ab
4	*	* is stronger than +, push *	+ *	ab
5	c	Operand, send to output	+ *	abc
End	—	Pop *, then pop +	empty	abc*+

Final answer:

a+b*c -> abc*+

Why?

Because $b*c$ must happen before $a+...$.

16. Example: $a*b+c$

Convert:

$a*b+c$

Expected postfix:

$ab*c+$

Dry run:

Step	Symbol	Action	Stack	Postfix
1	a	Operand, send to output	empty	a
2	*	Stack empty, push *	*	a
3	b	Operand, send to output	*	ab
4	+	* is stronger than +, pop *, then push +	+	ab*
5	c	Operand, send to output	+	ab*c
End	—	Pop +	empty	ab*c+

Final answer:

$a*b+c \rightarrow ab*c+$

17. Full Required Dry Run: $a+b*c-d/e$

Now we dry-run the main Lesson 15 example.

Expression:

$a+b*c-d/e$

Readable form:

a + b * c - d / e

Expected postfix:

abc*+de/-

We will write the stack with the top on the right.

Starting State

stack = empty
postfix = empty

Step-by-Step Trace

Step	Symbol	Action	Stack	Postfix
1	a	a is an operand, so send it directly to postfix.	empty	a
2	+	+ is an operator. Stack is empty, so push +.	+	a
3	b	b is an operand, so send it directly to postfix.	+	ab
4	*	* is stronger than +, so push *.	+ *	ab
5	c	c is an operand, so send it directly to postfix.	+ *	abc
6	-	- is weaker than *, so pop *. Then - has equal precedence with +, so pop +. Then push -.	-	abc*+
7	d	d is an operand, so send it directly to postfix.	-	abc*+d
8	/	/ is stronger than -, so push /.	- /	abc*+d
9	e	e is an operand, so send it directly to postfix.	- /	abc*+de
End	—	Pop remaining operators: first /, then -.	empty	abc*+de/-

Final answer:

a+b*c-d/e -> abc*+de/-

18. Explaining the Important Step in the Dry Run

The most important step is when we read `-`.

At that moment:

```
postfix = abc
stack = + *
current symbol = -
```

The stack top is `*`.

Compare:

```
* has precedence 2
- has precedence 1
```

Since `*` is stronger than `-`, pop `*` into postfix.

Now:

```
postfix = abc*
stack = +
```

Now compare stack top `+` with current symbol `-`.

```
+ has precedence 1
- has precedence 1
```

They are equal.

Since `+` and `-` are left-to-right operators, we pop `+` too.

Now:

```
postfix = abc*+
stack = empty
```

Finally, push `-`:

```
stack = -
```

This is why the postfix becomes:

```
abc*+
```

before continuing with `d/e`.

19. What the Stack Is Really Doing

The stack is not storing the final answer.

The stack stores operators that are waiting.

Example:

```
a + b * c
```

When we see `+`, we cannot immediately send it to postfix because a stronger operator may come later.

Then `*` comes later.

Since `*` is stronger, `+` must wait.

The stack allows this waiting.

Simple meaning:

```
The stack delays operators until it is safe to output them.
```

20. Beginner C++ Implementation

This beginner version handles:

single-character operands
operators +, -, *, /
spaces

This version does not yet handle:

parentheses
multi-digit numbers
unary operators
power operator
functions like log

Those can be added later.

Complete C++ Code

```
#include <iostream>
#include <stack>
#include <string>
#include <cctype>
using namespace std;

int precedence(char op) {
    if (op == '+' || op == '-') {
        return 1;
    }
    if (op == '*' || op == '/') {
        return 2;
    }
    return 0;
}

bool isOperand(char ch) {
    return isalnum(ch); // true for letters and digits
}

string infixToPostfix(string infix) {
    stack<char> st;
    string postfix = "";

    for (char sym : infix) {
        if (sym == ' ') {
            continue;
        }
        if (isOperand(sym)) {
            postfix += sym;
        }
        else {
            while (!st.empty() && precedence(st.top()) >= precedence(sym)) {
                postfix += st.top();
                st.pop();
            }
            st.push(sym);
        }
    }
    while (!st.empty()) {
        postfix += st.top();
        st.pop();
    }
    return postfix;
}
```

```

    }

    if (isOperand(sym)) {
        postfix += sym;
    } else {
        while (!st.empty() && precedence(st.top()) >= precedence(sym)) {
            postfix += st.top();
            st.pop();
        }

        st.push(sym);
    }
}

while (!st.empty()) {
    postfix += st.top();
    st.pop();
}

return postfix;
}

int main() {
    string infix = "a+b*c-d/e";

    cout << "Infix: " << infix << endl;
    cout << "Postfix: " << infixToPostfix(infix) << endl;

    return 0;
}

```

Output

```

Infix: a+b*c-d/e
Postfix: abc*+de/-

```

21. Code Explanation Step by Step

21.1 Header Files

```

#include <iostream>
#include <stack>

```

```
#include <string>
#include <cctype>
```

Meaning:

Header	Purpose
<code><iostream></code>	Used for <code>cout</code> .
<code><stack></code>	Used for the C++ stack container.
<code><string></code>	Used for <code>string</code> .
<code><cctype></code>	Used for <code>isalnum()</code> .

21.2 precedence() Function

```
int precedence(char op) {
    if (op == '+' || op == '-') {
        return 1;
    }
    if (op == '*' || op == '/') {
        return 2;
    }
    return 0;
}
```

This function gives a priority number to each operator.

Rules:

```
+ and - have priority 1
* and / have priority 2
```

So:

```
* and / are stronger than + and -
```

If the character is not one of these operators, the function returns `0` .

21.3 isOperand() Function

```
bool isOperand(char ch) {  
    return isalnum(ch);  
}
```

This checks whether a character is an operand.

`isalnum(ch)` returns true if `ch` is:

a letter or a digit

Examples that return true:

a, b, C, x, 5, 9

Examples that return false:

+, -, *, /

21.4 Creating the Stack and Output String

```
stack<char> st;  
string postfix = "";
```

`st` stores operators temporarily.

`postfix` stores the final answer.

21.5 Scanning the Expression

```
for (char sym : infix)
```

This loop reads each character from the infix expression.

Example:

a+b*c

The loop reads:

a
+
b
*
c

one by one.

21.6 Ignoring Spaces

```
if (sym == ' ') {  
    continue;  
}
```

If the expression contains spaces, the program ignores them.

So these two expressions are treated the same:

a+b*c

a + b * c

21.7 Handling Operands

```
if (isOperand(sym)) {  
    postfix += sym;  
}
```

If the symbol is an operand, add it directly to postfix.

Example:


```
sym = a
postfix = a
```

21.8 Handling Operators

```
else {
    while (!st.empty() && precedence(st.top()) >= precedence(sym)) {
        postfix += st.top();
        st.pop();
    }

    st.push(sym);
}
```

If the symbol is an operator, the program checks the operator stack.

It keeps popping from the stack while the stack top is stronger or equal.

Then it pushes the current operator.

21.9 Emptying the Stack at the End

```
while (!st.empty()) {
    postfix += st.top();
    st.pop();
}
```

After the full expression is scanned, some operators may still be in the stack.

They must be moved to postfix.

Example:

```
stack = - /
postfix = abc*+de
```

Pop / :

```
postfix = abc**de/
```

Pop ☐ - :

```
postfix = abc**de/-
```

22. Limitations of This Beginner Code

The beginner code is useful, but it is not a complete expression converter yet.

It handles:

```
single-character operands  
+, -, *, /  
spaces
```

It does not handle:

```
parentheses like (a+b)*c  
multi-digit numbers like 25+30  
unary minus like -a+b  
power operator like a^b  
functions like log(x)  
invalid expressions
```

Examples that need more advanced logic:

```
(a+b)*c  
25+30*4  
-a+b  
a^b^c  
logN
```

This is normal for Lesson 15 because the main goal is to understand the stack conversion idea first.

23. Method 2: All Symbols Pass Through the Stack

Lesson 15 also mentions a second method.

In Method 2, even operands can pass through the stack.

This method treats operands as if they have the highest precedence.

Example precedence idea:

Symbol type	Precedence
Operand	3
* /	2
+ -	1

So operands are considered stronger than all operators.

This method is clever, but it is usually harder for beginners.

Why?

Because beginners may get confused when operands are pushed onto the stack.

Method 1 is easier:

Operands go directly to output.
Operators go into the stack.

For now, focus on Method 1.

24. Method 1 vs Method 2

Idea	Method 1	Method 2
Operands	Go directly to postfix	May pass through stack
Operators	Use stack	Use stack
Beginner difficulty	Easier	Harder
Main rule	Operand direct, operator stack	All symbols handled by precedence
Recommended for beginners	Yes	Not first choice

Important beginner recommendation:

Use Method 1 first.

25. Complexity Analysis

Let n be the number of characters in the infix expression.

Time Complexity

The time complexity is:

$O(n)$

Why?

Because each symbol is scanned once.

Each operator is pushed onto the stack at most once.

Each operator is popped from the stack at most once.

So the total work grows linearly with the expression length.

Space Complexity

The space complexity is:

$O(n)$

Why?

Because the postfix output may store up to n characters.

The operator stack may also store up to n characters in the worst case.

So the required extra space grows with the expression length.

26. Common Beginner Mistakes

Mistake 1: Sending operators directly to postfix too early

Wrong idea:

When I see +, immediately put it in postfix.

Why this is wrong:

A stronger operator may come later.

Example:

a+b*c

The **+** appears before *****, but ***** must be done first.

So **+** must wait in the stack.

Mistake 2: Forgetting to pop remaining operators at the end

Expression:

a+b

After scanning, the stack still has **+**.

If you forget to pop it, your postfix becomes:

a b

Wrong.

Correct postfix:

ab+

Mistake 3: Using `>` instead of `>=`

For left-to-right operators, equal precedence must also cause popping.

Expression:

`a-b-c`

Correct grouping:

`(a-b)-c`

Correct postfix:

`ab-c-`

Using only `>` may produce wrong behavior for equal-precedence operators.

Mistake 4: Confusing stack output with final output

The stack is only temporary.

The postfix string is the final result.

Simple distinction:

```
stack = waiting operators
postfix = final answer being built
```

Mistake 5: Thinking the beginner code handles all expressions

The beginner code does not yet handle parentheses or multi-digit numbers.

Example:

`(a+b)*c`

This needs extra rules for `(` and `)`.

Lesson 15 focuses mainly on simple expressions without parentheses.

27. Practice Problems

Try converting these by hand.

Problem 1

$a+b*c$

Answer:

$abc*+$

Problem 2

$a*b+c$

Answer:

$ab*c+$

Problem 3

$a+b-c$

Answer:

$ab+c-$

Problem 4

$a*b/c$

Answer:

$ab*c/$

Problem 5

$a+b*c-d$

Answer:

$abc*+d-$

Problem 6

$a+b*c-d/e$

Answer:

$abc*+de/-$

28. Mini Revision Table

Concept	Simple meaning
Infix	Operator is between operands.
Postfix	Operator is after operands.
Operand	Value or variable, such as <code>a</code> , <code>b</code> , <code>5</code> .
Operator	Symbol such as <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> .
Precedence	Operator priority.
Stack purpose	Temporarily stores operators.
Operand rule	Send directly to postfix.
Operator rule	Compare precedence with stack top.

Concept	Simple meaning
Equal precedence	Pop first for left-to-right operators.
End rule	Pop all remaining stack operators.
Time complexity	$O(n)$.
Space complexity	$O(n)$.

29. Final Lesson 15 Summary

Lesson 15 teaches how to convert an expression from infix notation to postfix notation using a stack.

The core idea is:

Operands go directly to postfix.
Operators go into the stack until it is safe to output them.

The stack helps us respect precedence.

For example:

$a+b*c$

becomes:

$abc*+$

because multiplication happens before addition.

The most important algorithm rule is:

If current symbol is an operand:
add it to postfix.

If current symbol is an operator:
pop stack operators while they have greater or equal precedence.
then push the current operator.

At the end:
pop all remaining operators.

For the main example:

$a+b*c-d/e$

The correct postfix expression is:

$abc*+de/-$

That is the key result of Lesson 15.

30. What You Should Be Able to Do After This Lesson

After Lesson 15, you should be able to:

- Explain why postfix is useful.
- Identify operands and operators in an expression.
- Apply precedence rules for $+$, $-$, $*$, and $/$.
- Use a stack to temporarily store operators.
- Convert simple infix expressions to postfix by hand.
- Dry-run $a+b*c-d/e$ step by step.
- Understand the beginner C++ implementation.
- Explain why the algorithm is $O(n)$ time.
- Explain why the algorithm is $O(n)$ space.

If you can convert this expression correctly:

$a+b*c-d/e$

into:

$abc*+de/-$

and explain every stack movement, then you understand the main idea of Lesson 15.